Dare al modello dei documenti da leggere

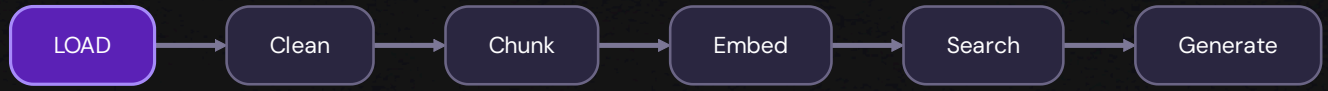
Data Loading — portare PDF, DOCX, HTML e Markdown dentro un'app AI

Modulo M2 · Lezione 16 · 30 giugno 2026



Dove siamo: la stage LOAD

Le lezioni del modulo non sono argomenti slegati: sono i mattoni di **un'unica pipeline** che culmina nel progetto RAG. Oggi costruiamo il primo pezzo — **portare i dati dentro**.



Finora (L13–L15) l'input lo scrivevamo **a mano** nel codice. Ma i dati veri vivono nei file: report, contratti, pagine web, appunti. **Il modello ragiona solo su ciò che gli diamo** — quindi prima dobbiamo saperli leggere.

> Oggi scriviamo due cose: un modulo `loaders/` che legge i file, e un piccolo `ask()` per **verificare** che il modello li veda davvero.

1. Tanti formati, un solo modello



Il modello vede solo ciò che gli diamo

Un LLM non "apre" un PDF. Riceve **testo** nel prompt e basta. Tutto il lavoro di oggi sta tra il file sul disco e quel testo: **estrarlo** in modo che il modello lo possa leggere.

📄 Il file

PDF, DOCX, HTML, Markdown. Ognuno è un contenitore con le sue regole — non è "solo testo".

🔧 Il loader

Il pezzo che apre il contenitore e ne **tira fuori il testo** (e qualche informazione su *da dove* arriva).

🤖 Il modello

Riceve quel testo nel prompt. Se l'estrazione è sporca o incompleta, **risponde peggio**.

"Garbage in, garbage out" vale doppio per un LLM: la qualità della risposta è limitata dalla qualità del testo che gli arriva. Per questo un intero modulo parte da qui.



Ogni formato è una bestia diversa

La stessa frase — "il fatturato 2025 è cresciuto del 12%" — può vivere in quattro contenitori completamente diversi. Stessa informazione, **tecnologie incompatibili** per tirarla fuori.

`PDF`

Non è testo lineare: è un **layout** di blocchi posizionati. Colonne, tabelle e a-capo vanno ricostruiti. A volte è solo un'immagine scansionata.

`DOCX`

Un archivio **zip di XML**. Il testo c'è, ma annegato in tag di formattazione, stili, revisioni.

`HTML`

Un **albero di tag** dove il contenuto utile è circondato da menu, footer, banner, script — il *noise*.

`Markdown`

Il più gentile: testo + una **sintassi leggera** (#, *, link). Quasi pronto, ma la sintassi va gestita.



Un formato interno unico

Non partiamo dalla soluzione, partiamo dal **dolore**. Se ogni loader restituisse una forma diversa…

❌ Senza formato comune

Il loader PDF torna stringhe, l'HTML torna un albero, il DOCX una lista di paragrafi. **Ogni** stage a valle — clean, chunk, embed — dovrebbe gestire tutti i casi, uno per uno. La conoscenza del formato si **spande** in tutta la pipeline.

✅ Con un `Document` unico

Ogni loader normalizza nello **stesso** contenitore: testo + metadata. Da lì in poi, nessuno sa più (né deve sapere) se veniva da un PDF o da una pagina web. Il formato si ferma **qui**.

Non è astrazione per il gusto di farla: è la barriera che impedisce all'eterogeneità di propagarsi. È lo stesso modello su cui convergono **tutte** le librerie del settore.



Il `Document`: testo + metadata

La soluzione minima è una `dataclass` piccolissima. Due campi: il **testo** estratto e un dizionario di **metadata**. Niente di più — più è sottile, meno somiglia a un framework.

```python
from dataclasses import dataclass, field

@dataclass
class Document:
    text: str                                  # il contenuto estratto
    metadata: dict = field(default_factory=dict)  # da dove arriva
```

`scripts/loaders/base.py`

Perché una `dataclass` e non un `dict`

Type hint, autocomplete, e un posto naturale dove appendere validazione più avanti — a costo quasi zero.

È il pezzo che attraversa tutto

Ogni stage del modulo riceve e produce `Document`. Lo costruiamo una volta, lo riusiamo fino al progetto.



I metadata non sono decorazione

Insieme al testo teniamo **da dove arriva**. Sembra un dettaglio oggi, ma sono i campi che fanno funzionare le lezioni dopo — vale la pena raccoglierli *mentre* carichiamo, non dopo.

`source`

Il file/URL di origine. Servirà per le **citazioni**: "questa risposta viene da *contratto.pdf*".

`page`

La pagina (sui PDF). Per puntare al punto esatto e per il **debug** del retrieval.

`type`

Il formato d'origine. Servirà per **filtrare** le ricerche nel vector DB (solo PDF, solo pagine web…).

> Citazioni e grounding torneranno in **L23** (prompt per RAG); il filtering per metadata in **L21** (vector DB). I campi si raccolgono adesso, gratis.



2. Caricare davvero



Il codice di oggi

Tutto quello che scriviamo dal vivo vive in `scripts/` : un file per formato dentro `loaders/` , più `ask.py` — il banco di prova con cui facciamo le prove. I documenti veri stanno in `sample_docs/` .

```
scripts/
├─ loaders/              un file per formato (lo costruiamo dal vivo)
│   ├─ base.py           Document (text + metadata)
│   ├─ pdf.py · docx.py · html.py · markdown.py
│   └─ dispatch.py       load()  ←  la porta unica
├─ ask.py                il banco di prova (riusa il client di L15)
└─ sample_docs/          documenti veri, un formato ciascuno
```

Come si prova

`python ask.py <file>` — carica e mostra il testo, **senza chiave**.
`python ask.py <file> "domanda"` — + la risposta del modello (serve la chiave).

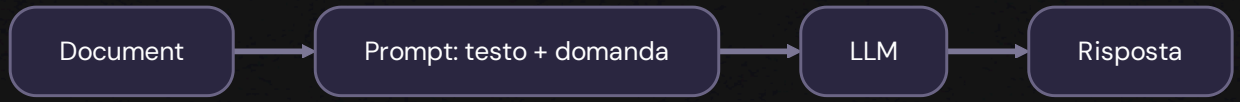
Un file per formato

Spezzare per formato rende ovvio dove si aggiunge il prossimo loader: un modulo nuovo + una riga in `load()` .



Il banco di prova: chiedere al modello

Come sappiamo se un loader funziona *davvero*? Non basta vedere del testo: deve essere testo che il **modello capisce**. Allora, dopo ogni loader, gli facciamo una domanda sul documento.



Riusiamo il `llm_client` del chatbot di L15: `ask(doc, "di cosa parla?")` infila il testo del documento nel prompt e gira la domanda al modello. È il nostro **strumento di verifica** per il resto della lezione.

`scripts/ask.py · provalo con: python ask.py <file> "domanda"`

⚠️ `ask()` è volutamente **naive**: ficca *tutto* il testo nel prompt, niente ricerca. È la **baseline** — funziona ora, ma a fine lezione la vedremo crollare. Quel crollo è il motivo per cui esistono embeddings, search e chunking.



Una libreria per ogni formato

Niente framework: per ogni formato una libreria **nativa**, quella che fa una cosa e la fa bene. Le incolliamo dietro la stessa interfaccia `Document`.

Formato	Libreria	Cosa fa	Dove fatica
PDF	`pypdf`	estrae testo pagina per pagina	tabelle, colonne, scansioni
DOCX	`python-docx`	legge paragrafi e tabelle dall'XML	layout complessi, caselle di testo
HTML	`BeautifulSoup`	naviga l'albero, estrae il testo	separare contenuto dal *noise*
Markdown	`markdown-it-py`	parse della sintassi → testo	— (è già quasi pulito)

Nessuna di queste "capisce" il documento: lo **parsa**. Regge bene il testo lineare; sui documenti complessi mostra i suoi limiti — ci torniamo alla fine. `(un file per formato in scripts/loaders/)`



`load()`: un'unica porta d'ingresso

Chi carica un documento non deve sapere quale libreria serve per quale file. Una sola funzione **smista** per estensione e restituisce sempre dei `Document`.

```python
from pathlib import Path

def load(path: str) -> list[Document]:
    ext = Path(path).suffix.lower()
    if ext == ".pdf":   return load_pdf(path)
    if ext == ".docx":  return load_docx(path)
    if ext in (".html", ".htm"): return load_html(path)
    if ext in (".md", ".markdown"): return load_markdown(path)
    raise ValueError(f"Formato non supportato: {ext}")
```

`scripts/loaders/dispatch.py · scripts/loaders/__init__.py`

Un **dispatcher**: l'interfaccia pubblica è `load(path)`, dietro ci sono i loader specifici. Aggiungere un formato = aggiungere una riga e una funzione. Lo riempiamo insieme, dal vivo.



PDF → `pypdf`

Il caso più comune e più infido. Estraiamo il testo **pagina per pagina** — così possiamo mettere il numero di pagina nei metadata.

```python
from pypdf import PdfReader

def load_pdf(path: str) -> list[Document]:
    reader = PdfReader(path)
    docs = []
    for i, page in enumerate(reader.pages):
        text = page.extract_text() or ""
        docs.append(Document(text, {"source": path, "page": i + 1, "type": "pdf"}))
    return docs
```

scripts/loaders/pdf.py

🤖 **Banco di prova:** `python ask.py sample_docs/bilancio.pdf "di quanto è cresciuto il fatturato?"` → il modello risponde dal PDF. **Funziona.**



DOCX → `python-docx`

Il testo è nei paragrafi dell'XML. Li leggiamo in ordine e li uniamo — un documento Word di solito è un unico `Document`.

```python
import docx

def load_docx(path: str) -> list[Document]:
    doc = docx.Document(path)
    text = "\n".join(p.text for p in doc.paragraphs if p.text)
    return [Document(text, {"source": path, "type": "docx"})]
```

scripts/loaders/docx.py

🤖 Banco di prova

Stesso `ask()` di prima, su un file diverso — un CV in Word: *"che ruolo ricopre?"*. Cambia solo il loader dietro: **l'interfaccia è identica**.

Le tabelle

`doc.tables` le espone a parte: estraibili, ma il layout va ricostruito a mano. Bandierina per i casi difficili.



HTML → `BeautifulSoup`: attenti al rumore

Qui il loader "funziona" ma la risposta **peggiora**. Una pagina web è piena di menu, footer, banner: se prendiamo tutto il testo, il *noise* finisce nel prompt e confonde il modello.

```python
from bs4 import BeautifulSoup

def load_html(path: str) -> list[Document]:
    soup = BeautifulSoup(open(path, encoding="utf-8"), "html.parser")
    for tag in soup(["script", "style", "nav", "footer"]):  # via il noise più ovvio
        tag.decompose()
    text = soup.get_text(separator="\n", strip=True)
    return [Document(text, {"source": path, "type": "html"})]
```

`scripts/loaders/html.py` · per la demo, commenta la riga `decompose()` e confronta

🤖 `ask("quali ingredienti servono?")` sulla ricetta: con tutto il testo il modello ci infila anche il menu e la newsletter; tolti `nav` / `footer` / `script`, risponde pulito. Ma "pulire bene" è un mestiere a sé → è **esattamente la prossima lezione** (L17, Text Cleaning).



Markdown → testo + struttura

Il formato più gentile: è già testo. Lo parsiamo solo per gestire la sintassi (header, link, code) in modo pulito invece di lasciarla grezza.

```python
from markdown_it import MarkdownIt
from bs4 import BeautifulSoup

def load_markdown(path: str) -> list[Document]:
    md = open(path, encoding="utf-8").read()
    html = MarkdownIt().render(md)
    text = BeautifulSoup(html, "html.parser").get_text("\n", strip=True)
    return [Document(text, {"source": path, "type": "markdown"})]
```

`scripts/loaders/markdown.py · prova: python ask.py sample_docs/readme.md`

Quattro formati, **quattro loader**, una sola interfaccia `Document`. Il `load()` di prima ora li smista tutti — e `ask()` funziona uguale su ognuno.



Tocca a voi: un loader per `.txt` / `.csv`

(20') Avete visto il pattern quattro volte. Ora lo aggiungete voi: due formati nuovi, stessa interfaccia.

Cosa fare

Scrivete `load_txt` e `load_csv` che tornano dei `Document` coi giusti metadata, e agganciateli al dispatcher `load()`. Per il CSV, decidete voi: una riga = un Document, o tutto insieme?

Come verificare

Provate su `sample_docs/note.txt` e `clienti.csv` (o un vostro file) con lo stesso `ask()`. Se la risposta ha senso, il loader è a posto.

Da qui in poi, ogni formato nuovo per voi è solo "un loader in più dietro la stessa porta".

`stub: scripts/esercizio/txt_csv.py · soluzione: esercizio/soluzione_txt_csv.py`



3. I limiti (e perché c’è un modulo intero, dopo)



Il muro: tutto nel prompt non scala

Il nostro `ask()` ficca l'**intero** documento nel prompt. Va benissimo per un volantino di una pagina. Proviamo con un PDF da 300 pagine, o con 500 documenti...

📄 Un documento corto

Ci sta nel **context window**, costa pochi centesimi. `ask()` funziona. Tutto bene.

📚 Tanti documenti lunghi

Non ci stanno nel contesto (L10), e anche dove ci stessero pagheresti una fortuna a ogni domanda (L14). Il muro.

> La domanda giusta non è "come ci infilo tutto", ma **"come do al modello solo i pezzi che servono"**. È la domanda a cui risponde il resto del modulo.



Ecco perché: embed → search → chunk

Caricare i documenti è il **primo** passo. Il muro che abbiamo appena toccato è la motivazione di tutto quello che viene dopo: dare al modello **solo i pezzi rilevanti**, non tutto.



Spezzare i documenti (**chunk**), trasformarli in vettori (**embed**), cercare i pezzi giusti (**search**) e tenerli in un **vector DB**. Non oggi — ma adesso sapete *perché* servono.

Quando il parsing nativo si rompe

C'è un secondo limite, sui documenti **difficili**. Le librerie native reggono il testo lineare; su PDF con tabelle complesse, colonne o pagine scansionate vanno in crisi: testo a pezzi, tabelle distrutte.

🔧 Parsing locale (oggi)

Gratis, privato, offline. Perfetto per la maggior parte dei documenti. Si rompe sul **layout complesso**.

☁️ Parser gestiti (es. LlamaParse)

A pagamento, mandano i doc nel cloud, ma **robusti** su tabelle e scansioni. Si agganciano come "un loader in più".

🚩 Solo una bandierina per oggi: il trade-off **locale vs gestito** lo proviamo con le mani al Lab del sabato (**L18**), su dati reali sporchi.



Grazie

Quattro formati, un `Document`, una porta sola: `load()`

Ora il modello può leggere i nostri file — ma il testo è ancora **sporco**. Prossimo passo: ripulirlo, **Text Cleaning & Pre-processing** (L17).

